# Contents

# 1 Computers dataset

We will look at a computers dataset provided in `computers.csv`, which contains 6,259 observations of 10 variables. To check if there are any NA values in the dataset, we can use `anyNA()`. Running it here indicates that there were no missing values in the dataset.

```
computers <- read.csv('computers.csv', header=T)
str(computers)
```

```
## 'data.frame':    6259 obs. of  10 variables:
##  $ price  : int  1499 1795 1595 1849 3295 3695 1720 1995 2225 2575 ...
##  $ speed  : int  25 33 25 25 33 66 25 50 50 50 ...
##  $ hd     : int  80 85 170 170 340 340 170 85 210 210 ...
##  $ ram    : int  4 2 4 8 16 16 4 2 8 4 ...
##  $ screen : int  14 14 15 14 14 14 14 14 14 15 ...
##  $ cd     : chr  "no" "no" "no" "no" ...
##  $ multi  : chr  "no" "no" "no" "no" ...
##  $ premium: chr  "yes" "yes" "yes" "no" ...
##  $ ads    : int  94 94 94 94 94 94 94 94 94 94 ...
##  $ trend  : int  1 1 1 1 1 1 1 1 1 1 ...
```

```
anyNA(computers)
```

```
## [1] FALSE
```

Looking briefly at the dataset, we can see the scale of each predictor is wildly different. Again, we will have to normalise the data so that one feature does not dominate in the distance measurement.

```
numvars <- sapply(computers, is.numeric)
computers1 <- computers[numvars]
summary(computers1)
```

```
##      price          speed             hd               ram        
##  Min.   : 949   Min.   : 25.00   Min.   :  80.0   Min.   : 2.000  
##  1st Qu.:1794   1st Qu.: 33.00   1st Qu.: 214.0   1st Qu.: 4.000  
##  Median :2144   Median : 50.00   Median : 340.0   Median : 8.000  
##  Mean   :2220   Mean   : 52.01   Mean   : 416.6   Mean   : 8.287  
##  3rd Qu.:2595   3rd Qu.: 66.00   3rd Qu.: 528.0   3rd Qu.: 8.000  
##  Max.   :5399   Max.   :100.00   Max.   :2100.0   Max.   :32.000  
##      screen           ads              trend      
##  Min.   :14.00   Min.   : 39.0   Min.   : 1.00  
##  1st Qu.:14.00   1st Qu.:162.5   1st Qu.:10.00
```

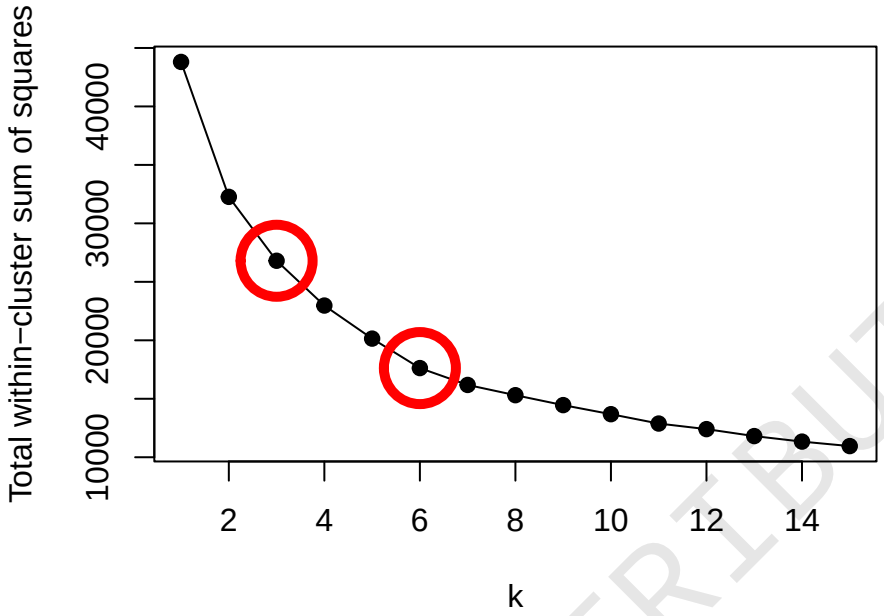
```
##  Median :14.00   Median :246.0   Median :16.00
##  Mean   :14.61   Mean   :221.3   Mean   :15.93
##  3rd Qu.:15.00   3rd Qu.:275.0   3rd Qu.:21.50
##  Max.   :17.00   Max.   :339.0   Max.   :35.00
```

```
computers.nor <- scale(computers1)
```

## 2 Performing k-means clustering

With the normalised dataset, we can focus on performing k-means with `kmeans()`. Again, we will iterate through 1 to 15 possible clusters to determine a good number of clusters. It is significantly harder to see a bend in the graph here - you may try to shrink/expand the x or y-axis with the `xlim` or `ylim` parameter in `plot()` to help in identifying the bend. The two candidate points I identify are $k = 3$ and $k = 6$.

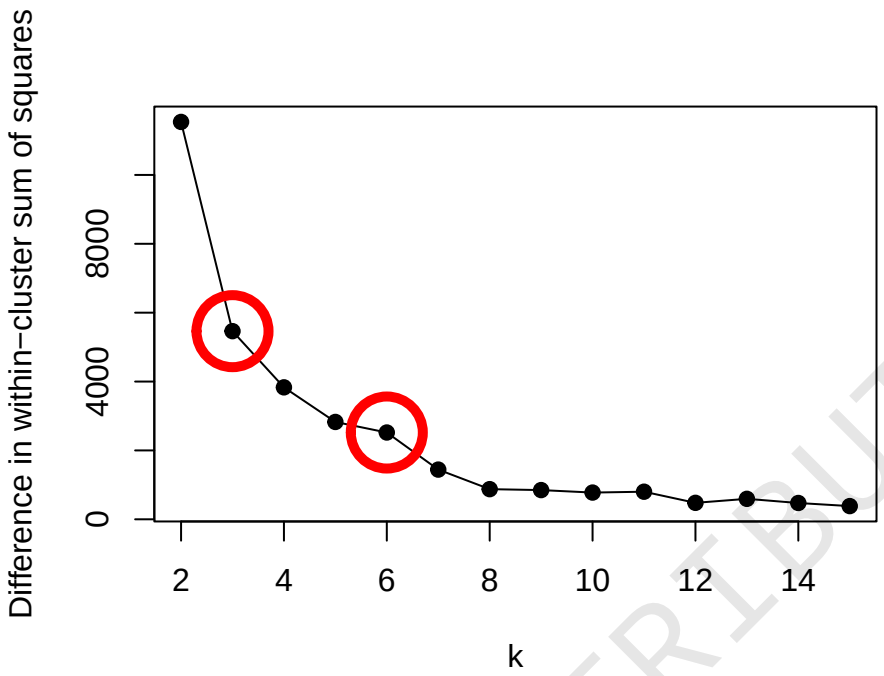
```
wcss <- function(k) {
  set.seed(100)
  kmeans(computers.nor, k, nstart = 10)$tot.withinss
}

wcss_k <- sapply(1:15, wcss)

plot(wcss_k, type="o", pch=19, xlab="k", ylab="Total within-cluster sum of
↪  squares")
k <- c(3, 6)
points(x=k, y=wcss_k[k], pch=1, cex=5, col="red", lwd=5)
```



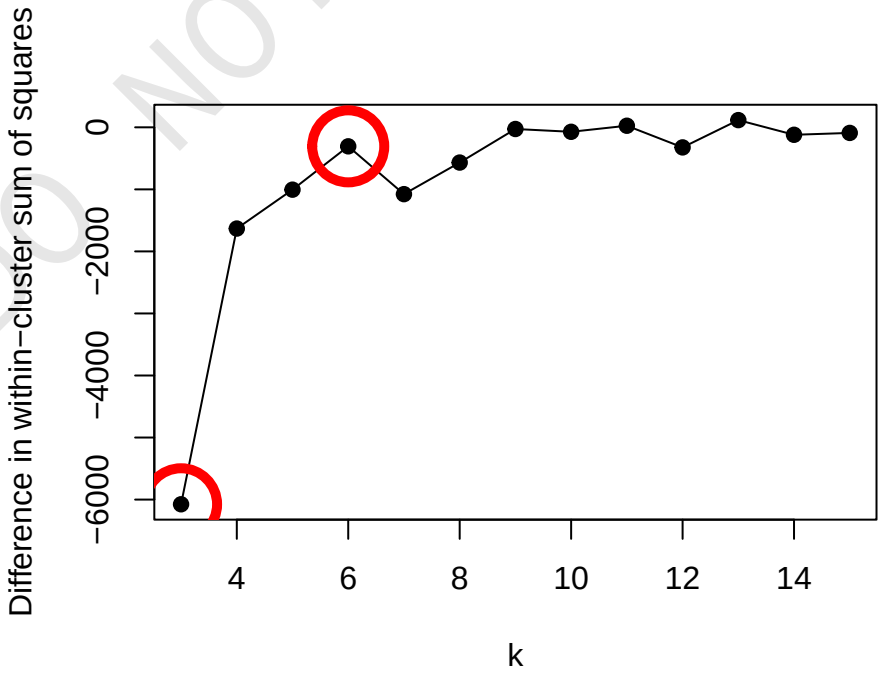

### 2.1 Extremely optional: extra criteria on selection of k

Looking at the differences and focusing on the second difference, $k = 6$ has the behaviour of a rise and a fall.

```
plot(2:15, -diff(wcss_k), type="o", pch=19, xlab="k", ylab="Difference in
↪  within-cluster sum of squares")
points(x=k, y=-diff(wcss_k)[k-1], pch=1, cex=5, col="red", lwd=5)
```




```r
plot(3:15, -diff(wcss_k, differences=2), type="o", pch=19, xlab="k",
↪  ylab="Difference in within-cluster sum of squares")
points(x=k, y=-diff(wcss_k, differences=2)[k-2], pch=1, cex=5, col="red",
↪  lwd=5)
```



### 2.2 Final k-means clustering

Proceeding on with a final choice of $k = 6$, we can examine the resulting clusters. With the clustering we have, we can see, for example, that cluster 4 may be high-end computers, with a high price, speed, HDD space and RAM. The characteristics of cluster 3 suggest they are lower-end computers with lower specifications across the board.

```
final.k <- 6
set.seed(100)
kmeans.final <- kmeans(computers.nor, final.k, nstart = 10)

computers1 %>% mutate(Cluster = kmeans.final$cluster) %>%
  group_by(Cluster) %>%
  summarise(across(everything(), mean), count=n())
```

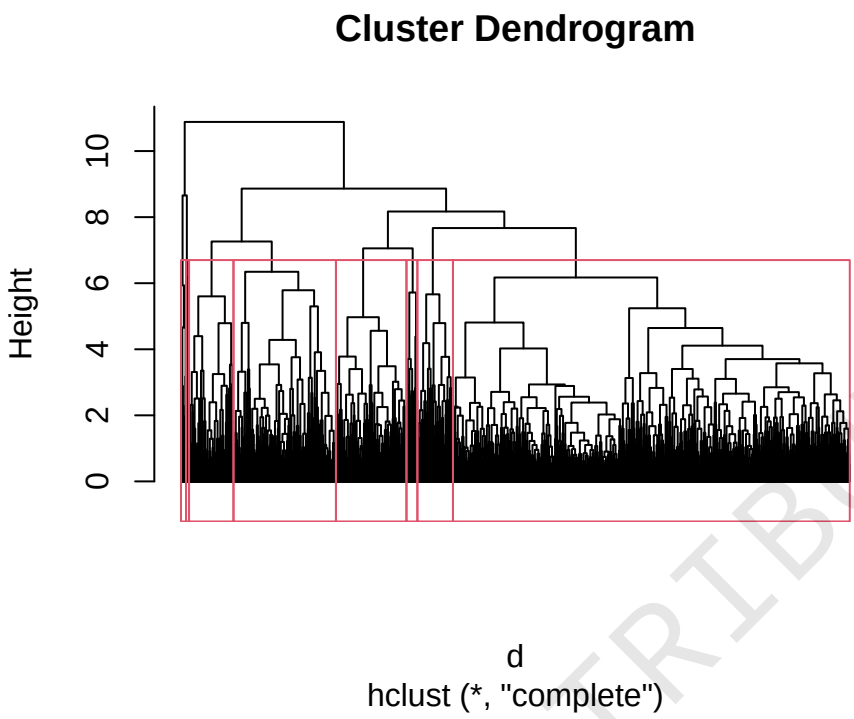
```
## # A tibble: 6 x 9
##   Cluster price speed    hd   ram screen   ads trend count
##     <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <int>
## 1       1 2574.  56.5  420.  7.95   17    223. 17.3    542
## 2       2 2723.  61.6  472. 11.0    14.4  274. 13.7   1318
## 3       3 1845.  38.1  241.  4.51   14.2  268. 13.2   2150
## 4       4 2726.  63.7  957. 20.3    15.1  151. 25.6    612
## 5       5 1779.  64.3  500.  6.67   14.3  155. 25.4   1014
## 6       6 2359.  44.4  236.  6.65   14.3  123.  3.74   623
```

## 3 Hierarchical clustering

The hierarchical clustering proceeds on as normal. Notice the omission of the `method` parameters, since in both functions we are using the default values.

The resulting dendrogram is a mess, with more than 6,000 entries.

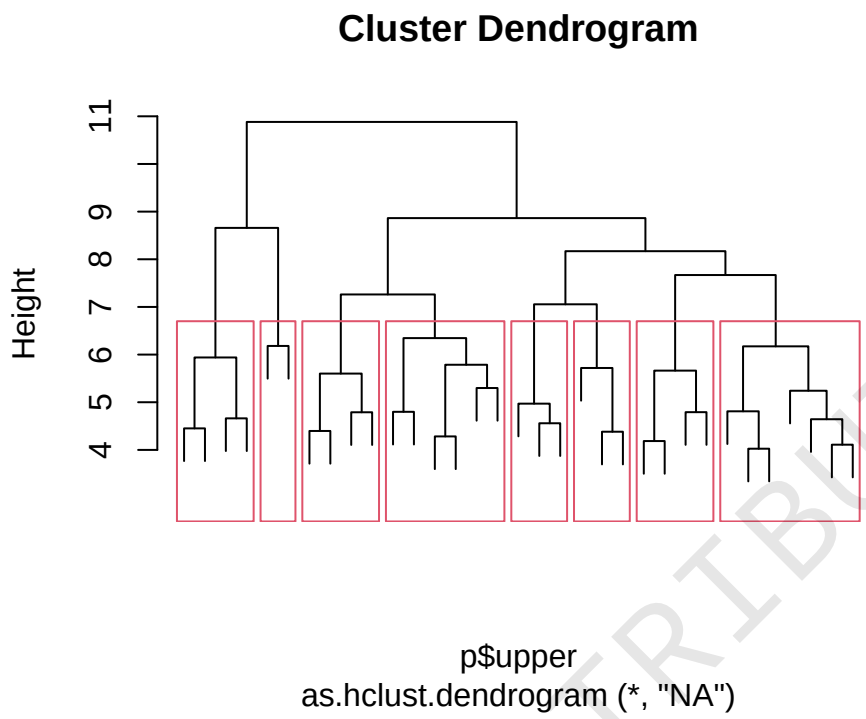
```
d <- dist(computers.nor)
hier.clust <- hclust(d)

plot(hier.clust, labels = F, hang = -1)
rect.hclust(hier.clust, k = 8)
```




### 3.1 Extremely optional: cleaning up the dendrogram

From the dendrogram, we may want to focus on the area above height 4. The `cut()` function can be used, which splits the dendrogram into an `upper` (height > 4) and `lower` (height < 4). Since we are interested in the upper part of the dendrogram, we can continue and plot it as usual. Note that the size of each dendrogram branch no longer conveys the size of that cluster, but it is easier to see the height difference when the clusters are combined.

```
p <- cut(as.dendrogram(hier.clust), h = 4)
p.plot <- as.hclust(p$upper)
plot(p.plot, labels = F, hang = 0.1)
rect.hclust(p.plot, k = 8)
```

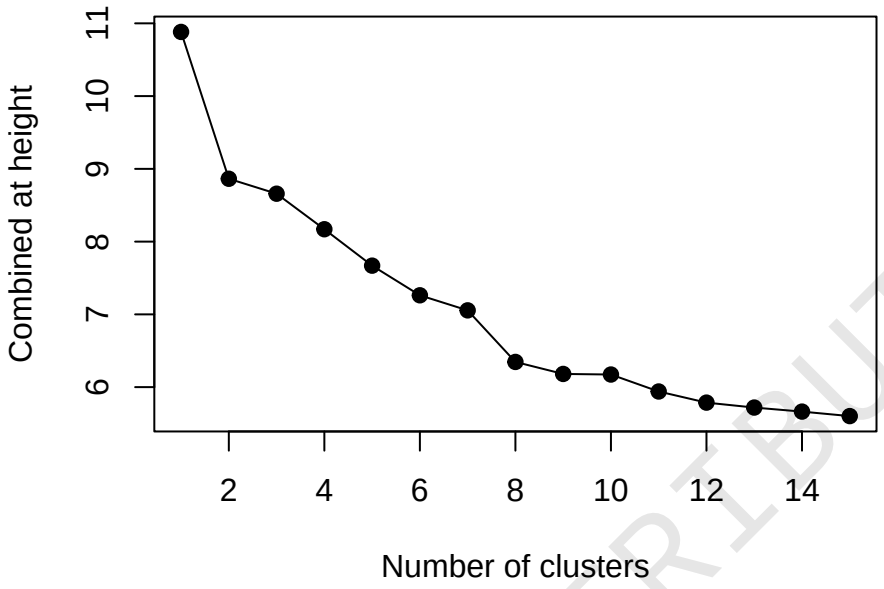



### 3.2 Extremely optional: visualising the height of combined clusters

The output of `hclust()` returns us an object with a `height` vector, which contains all heights at which two clusters were combined in ascending order. We can thus plot out the last few values to see the point at which there was a significant jump to identify candidate values of $k$. In the below graph, for example, the data was combined into one cluster at a height of around 11. In this case, the jump from 8 clusters to 7 clusters was significant, and so we can use this as a candidate value.

From the above dendrograms, you can also see the selection of $k = 8$ has a big jump as compared to $k = 7$ or $k = 9$.

```
h <- rev(hier.clust$height)
plot(head(h, 15), pch = 19, type = 'o',
     xlab = "Number of clusters",
     ylab = "Combined at height")
```




### 3.3 Extra: resulting clusters of hierarchical clustering

Notice how the number of clusters determined by k-means and hierarchical clustering were different - this is due to the difference in how each observation was assigned a cluster, and also how we determined the optimal number of clusters.

```
hier.clust.8 <- cutree(hier.clust, 8)

computers1 %>% mutate(Cluster = hier.clust.8) %>%
  group_by(Cluster) %>%
  summarise(across(everything(), mean), count = n())
```

```
## # A tibble: 8 x 9
##   Cluster price speed    hd   ram screen   ads trend count
##     <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <int>
## 1       1 2022.  45.0  304.  6.12   14.3 252.  13.7   3714
## 2       2 2925.  54.7  377. 10.6    14.5 190.   7.17   660
## 3       3 2612.  50.1  338.  6.98   17   253.  13.8    333
## 4       4 4030.  64.0  492. 16.3    17.0 246.  10.4     47
## 5       5 3288.  88.6  815. 15.7    14.3 267.  16.4    102
## 6       6 1898.  67.3  586.  8.28   14.9 142.  26.7    959
## 7       7 2771.  64.4  975. 21.1    15.1 151.  25.7    416
## 8       8 3022.  77.8 1525  26.6    15.8  61.5 32.6     28
```